LLM API Providers

A practical guide to the four providers powering this session — how they're priced, why we chose them, and how our code stays provider-agnostic across all of them.

The Four Providers in This Session

Provider	Model We Use	Why We Picked It
Groq	`llama-3.3-70b-versatile`	Fastest token throughput on the free tier — good for demo latency.
Google AI Studio	`gemini-2.5-flash-lite`	Free quota, solid factual recall, totally different family from Llama.
Cerebras	`llama3.1-8b`	Independent inference stack; diverges from Groq even on the same base model.
OpenRouter	`meta-llama/llama-3.2-3b-instruct`	Unified key → 100+ models; our Consolidator lives here.

Why Four Different Providers?

Using multiple providers allows us to leverage diverse strengths and maintain flexibility in our development.

The Researcher + Consolidator Pattern

We strategically utilize individual providers as "researchers" to explore specific model capabilities and performance characteristics, while a "consolidator" (like OpenRouter) provides a unified interface to abstract away vendor-specific details and offer a wider range of models.

Diverse Model Access

Tap into a broad spectrum of AI models, each with unique strengths for different tasks.

Mitigate Vendor Lock-in

Reduce dependency on a single provider, ensuring adaptability and long-term project viability.

Optimize Performance & Cost

Switch between providers to achieve the best balance of speed, accuracy, and expenditure.

How LLM APIs Are Priced

Free Tiers

Many providers offer generous free usage tiers, perfect for development, testing, and smaller-scale applications.

Paid Tier

Pricing models vary, often based on input/output tokens, compute time, or calls. Understanding these is key for scaling.

Rate Limits

APIs typically enforce limits on requests per minute or tokens per minute to prevent abuse and manage infrastructure load.

How Our Code Stays Provider-Agnostic

To ensure our application can seamlessly switch between different LLM providers without extensive code changes, we employ a strategy of abstraction. This involves creating a unified interface or wrapper that standardizes how we interact with various APIs. By doing so, the core logic of our application remains decoupled from the specific implementation details of each LLM provider.

This approach allows us to:

- Easily integrate new providers.
- Switch providers based on performance, cost, or feature availability.
- Maintain a consistent codebase, reducing complexity and potential for errors.

```
// Example of a provider-agnostic call
async function
generateResponse(modelName, prompt) {
  const provider =
getProviderForModel(modelName);
  if (!provider) {
    throw new Error(`No provider found
for model: ${modelName}`);
  }
  return await
provider.generate(prompt);
}

// Inferred provider mapping
(simplified)
function
getProviderForModel(modelName) {
  if (modelName.startsWith('llama'))
return groqProvider;
  if (modelName.startsWith('gemini'))
return googleProvider;
  // ... more logic to map models to
providers
}
```

What Is Not in Scope Today

⚠ While these advanced topics are crucial for some LLM applications, they are beyond the scope of this session, which focuses on foundational API usage.

Fine-tuning & LoRA

Techniques for customizing models with specific datasets to improve performance on specialized tasks.

Self-hosted Models

Running LLMs on your own infrastructure for full control over data, security, and inference.

Vendor-specific Features

Advanced functionalities unique to a particular provider, such as complex tool use or specialized data connectors.